Web Application Software Architecture:

Web application architecture defines the structure and interactions between components in a web application. Let's explore the key concepts:

1. Software Architecture Types

1.1 Single-Tier Architecture

In a **single-tier architecture**, the application's entire logic resides in one layer, typically on the client machine. It is simple but not scalable for larger applications.

Example:

● A desktop application where the database and user interface exist in the same system (e.g., Microsoft Access).

Advantages:

● Simple development and deployment.

● Fast data access as everything is on the same machine.

Disadvantages:

● Limited scalability.

● Difficult to maintain and update.

1.2 Two-Tier Architecture

In a **two-tier architecture**, the client and server are separated:

1. **Client Tier**: Contains the user interface.

2. **Server Tier**: Manages the database and business logic.

Example:

● A Java Swing application communicating with a MySQL database.

Advantages:

● Improved performance by splitting responsibilities.

● Easier to maintain than single-tier.

Disadvantages:

● Scalability is limited as the server becomes a bottleneck.

● Changes to business logic require redeployment of server code.

2. Communication Between Client and Server

2.1 How Communication Happens

● **Request-Response Model**: Clients send HTTP/HTTPS requests to servers, which respond with data or services.

● **Protocols**: ○ **HTTP/HTTPS**: Used for web pages and RESTful APIs.

○ **WebSockets**: For real-time communication.

○ **RPC (Remote Procedure Calls)**: For direct method invocation.

Example of HTTP Request-Response:

1. **Client**: Sends an HTTP GET request to https://example.com/api/data.

2. **Server**: Processes the request and responds with JSON data.

3.Monolithic vs. Microservices Architecture

3.1 Monolithic Architecture

In **monolithic architecture**, the entire application is built as a single unit. All functionalities, such as authentication, business logic, and database operations, are bundled together.

Advantages:

© 2026 GUVI GEEK NETWORK PVT LTD. All rights reserved. No part of this document may be reproduced or transmitted in any form or by any means, electronic, mechanical, photocopying, recording, or otherwise, without prior permission from GUVI.

● Simple to develop and deploy.

● Easy to debug as the application is in one codebase.

Disadvantages:

● Difficult to scale specific parts of the application.

● Changes in one module may affect the entire system.

● Slower development for large teams due to dependencies.

Diagram:

Single Application -> Database

3.2 Microservices Architecture

In **microservices architecture**, the application is divided into smaller, loosely coupled services. Each service handles a specific functionality and communicates via APIs.

Advantages:

● Scalability: Services can scale independently.

● Flexibility: Easier to adopt new technologies for specific services.

● Faster development with small, autonomous teams.

Disadvantages:

● More complex to deploy and manage.

● Communication between services can become a bottleneck.

Diagram:

Service A <--> Database A

Service B <--> Database B

Service C <--> Database C

4. What is a REST API?

4.1 Introduction to REST

REST (Representational State Transfer) is an architectural style for designing networked applications. It relies on stateless, client-server communication over HTTP.

Key Concepts:

1. **Resources**: Identified by URLs (e.g., /users, /products).

2. **HTTP Methods**: ○ **GET**: Retrieve data.

○ **POST**: Create new data.

○ **PUT**: Update existing data.

○ **DELETE**: Remove data.

3. **Statelessness**: Each request contains all the information needed to process it.

4. **Representation**: Data is usually represented in JSON or XML format.

Example REST API Endpoint:

● **Endpoint**: https://example.com/api/users

● **GET /api/users**: Fetch all users.

● **POST /api/users**: Create a new user.

5. Introduction to Java Containers & Servlets

5.1 Java Containers

A **Java container** provides the runtime environment for Java web applications. It manages the lifecycle of servlets, handles requests, and ensures compliance with the Java Servlet Specification.

Examples:

● **Apache Tomcat**: A lightweight servlet container.

● **Jetty**: Used for embedded applications.

● **GlassFish**: A full-fledged Java EE application server.

5.2 Servlets

A **servlet** is a Java class used to handle HTTP requests and generate dynamic web content.

Example:

```
import javax.servlet.*; import javax.servlet.http.*; import java.io.IOException; public class HelloWorldServlet extends HttpServlet { protected void doGet(HttpServletRequest request, HttpServletResponse response) throws IOException { response.setContentType("text/html"); response.getWriter().println("<h1>Hello, World!</h1>"); } }
```